Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Исследование рядов Тейлора»

Выполнила:
студентка группы 3825Б1ПМии1
Разина В.А.

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Введение 3
Постановка задачи 4
Руководство пользователя 5
Описание программной реализации 6
Результаты экспериментов 7
Заключение 10
Литература 11
Приложение 12

Введение

При написании программ часто приходится складывать числа, которые сильно отличаются по размеру. В таких случаях порядок сложения может влиять на конечный результат, это важно учитывать для точной работы программы. В работе наглядно показано, как меняется сумма при использовании разных способов сложения.

Постановка задачи

Разработаем консольное приложение на языке C, которое будет позволять пользователю наглядно видеть разную погрешность, возникающую при суммировании членов рядов Тейлора в прямом (от большего по модулю слагаемого к меньшему) и обратном (от меньшего к большему) порядке. Результаты суммирования могут отличаться.

Руководство пользователя

1. После запуска программы, вы увидите в консоли выбор из нескольких вариантов функций, а также вариант выхода из приложения *(см. рис 1)*.

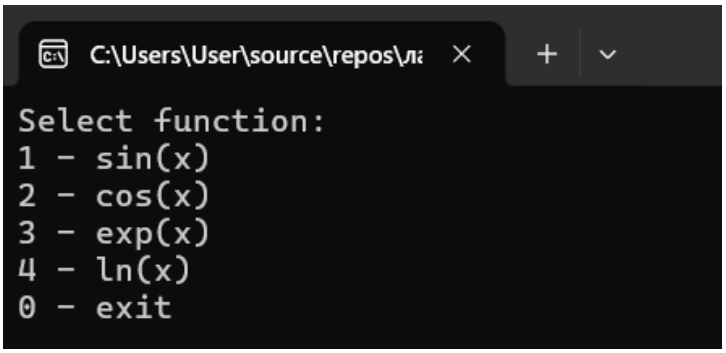


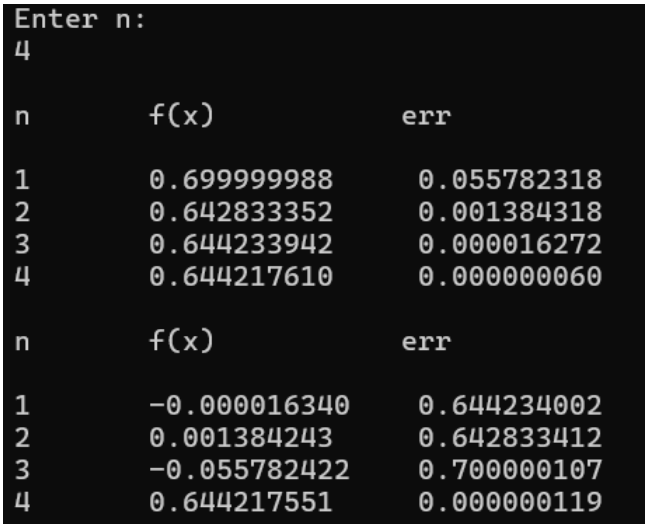
Рис. 1. Меню приложения.

Для выбора функции, нажмите на клавиатуре одну из клавиш 1,2,3,4:

1. sin(x);
2. cos(x);
3. exp(x);
4. ln(x).

Для выхода из приложения нажмите кнопку 0.

2. После выбора функции, будет предложено ввести количество слагаемых. Далее вы увидите на экране две таблицы (для прямого суммирования и обратного), в первом столбце которой записано количество слагаемых, во втором – значение функции, а в третьем – абсолютная погрешность *(см. рис 2)*.



```
Enter n:
4

n       f(x)           err

1       0.699999988     0.055782318
2       0.642833352     0.001384318
3       0.644233942     0.000016272
4       0.644217610     0.000000060

n       f(x)           err

1       -0.000016340    0.644234002
2       0.001384243     0.642833412
3       -0.055782422    0.700000107
4       0.644217551     0.000000119
```

Рис. 2. Таблица значение функции, погрешность

Далее программа снова предложит вам выбрать функцию или выйти.

Описание программной реализации

1) `void Point(int point)` - функция, которая обрабатывает выбор пользователя и вызывает остальные функции. Входящие параметры: номер функции.

2) `void start_menu()` – выводит текст основного меню в консоль.

3) `struct Teylor` – тип данных, который состоит из количества слагаемых в формуле Тейлора, массивов значений, частичных сумм и погрешностей.

4) `void init(Teylor *xn, int n)` – функция выделяющая память под массивы.

5) `void clean( Teylor* xn)` – функция для освобождения памяти.

6) Функции суммирования членов ряда Тейлора:

   a. `void sum_left(Teylor* xn)` суммирование в прямом порядке;

   b. `void sum_right( Teylor* xn)` суммирование в обратном порядке.

Заполняют массивы частичных сумм (значения функции, полученные с помощью формулы Тейлора при количестве слагаемых от 1 до n), складывая значения слева направо (прямой порядок) и справа налево (обратный).

7) `void print(Teylor xn)` – функция вывода массивов значений, на вод принимает переменную типа Тейлор.

8) `void fill_print(void(*func_Teylor)(Teylor* xn, float x, void(*sum)(Teylor* xn)), Teylor a,Teylor b, float x1, float x2, float(*func_math)(float x))` – функция, которая выводит значения функций (sin(x), cos(x), exp(x), ln(x)) для каждого количества слагаемых (от 1 до n) в формуле Тейлора и соответствующую абсолютную погрешность полученного значения. Входящие параметры: указатель на функцию вычисления ряда Тейлора, значение x, переменные типа Teylor (композитный тип данных), указатель на стандартную функцию из библиотеки `"math.h"`.

9) Функции вычисления рядов Тейлора:

   a. `void Teylor_sin(Teylor* xn, float x, void(*sum)(Teylor* xn))`

   b. `void Teylor_cos( Teylor* xn, float x, void(*sum)(Teylor* xn))`

   c. `void Teylor_exp( Teylor* xn, float x, void(*sum)(Teylor* xn))`

   d. `void Teylor_ln( Teylor* xn, float x, void(*sum)(Teylor* xn))`

Входящие параметры: указатель на структуру Teylor, значение x и указатель на функцию сложения. Заполняют массивы значений (члены ряда Тейлора), вычисляя их по рекуррентным формулам, вызывают функции заполнения массива частичных сумм `void sum_left(Teylor* xn)` или `void sum_right(Teylor* xn)`.

10) `void err_fill( Teylor* xn, float x, float(*func_math)(float))` – функция заполнения массива абсолютных погрешностей. Входящие параметры: переменная Teylor, значение x и указатель на функцию из библиотеки `"math.h"`.

Результаты экспериментов

Для каждой из четырех функций: sin(x), cos(x), exp(x) и ln(x) – было посчитано значение тремя способами (0,5≤x≤1):

1 С помощью встроенных функций из библиотеки `"math.h"`;

2 Используя разложение в ряд Тейлора и суммирование ряда в прямом порядке (от большего слагаемого к меньшему);

3 Используя разложение в ряд Тейлора и суммирование ряда в обратном порядке (от меньшего слагаемого к большему).

Запуски программы проводились в VisualStudio 2022.

Эталонным значением будем считать результат, полученный при использовании функций библиотеки `"math.h"` . Абсолютная погрешность равна модулю разности эталонного значения и значения, являющегося результатом разложения по формуле Тейлора.

$$\Delta = |f_{math} - f_{Teylor}|$$

Ниже представлены таблицы полученных результатов и графики зависимости абсолютной погрешности от количества слагаемых в ряде Тейлора для каждой функции.

1. Функция sin(x)

n, кол-во слагаемых	sin(x), суммы в прямом порядке	err, погрешность	sin(x), суммы в прямом порядке	err, погрешность
1	0.7000000	0.0557823	-0.0000000	0.6442177
2	0.6428334	0.0013843	0.0000000	0.6442177
3	0.6442339	0.0000163	-0.0000000	0.6442177
4	0.6442176	0.0000001	0.0000000	0.6442177
5	0.6442177	0.0000001	-0.0000000	0.6442177
6	0.6442177	0.0000001	0.0000001	0.6442176
7	0.6442177	0.0000001	-0.0000162	0.6442339
8	0.6442177	0.0000001	0.0013844	0.6428333
9	0.6442177	0.0000001	-0.0557823	0.7000000
10	0.6442177	0.0000001	0.6442177	0.0000000

Таблица 1. Значения и погрешности для sin(x).

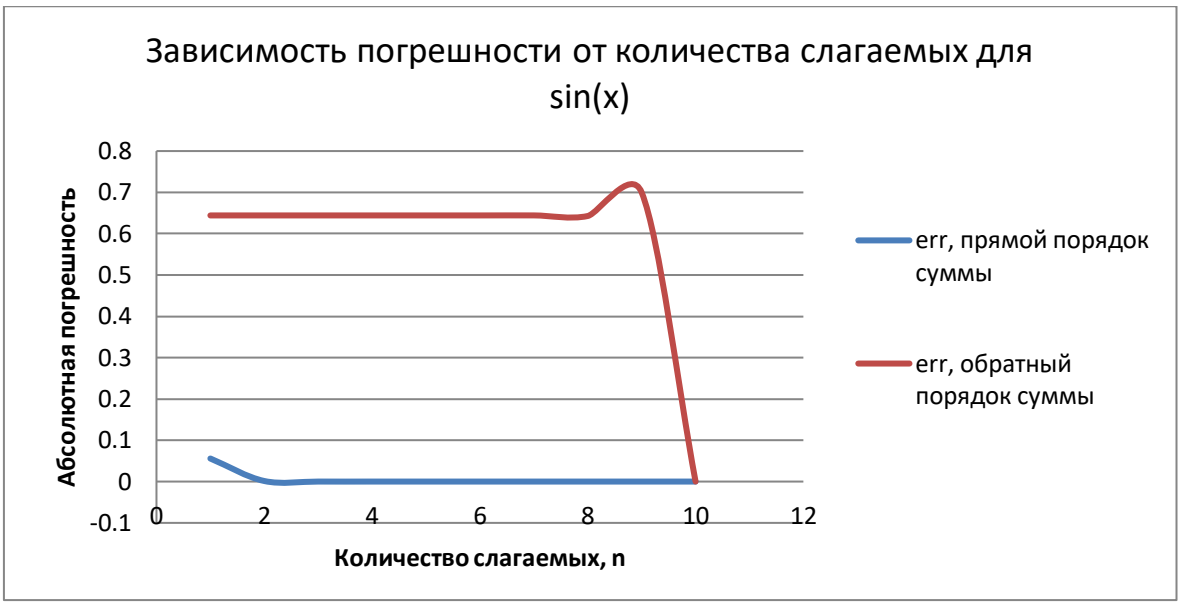


График 1. Зависимость погрешности от количества слагаемых для sin(x).

Прямой порядок суммирования членов ряда Тейлора (от наибольшего по модулю к наименьшему) для sin(x) обеспечивает быстрое уменьшение абсолютной погрешности, которая достигает минимума (порядка 10^{-7}) при небольшом количестве слагаемых. Обратный порядок суммирования (от наименьшего по модулю к наибольшему) приводит к накоплению более высокой погрешности, чем в первом случае.

2. Функция cos(x)

n, кол-во слагаемых	cos(x), суммы в прямом порядке	err, погрешность	cos(x), суммы в прямом порядке	err, погрешность
1	1.0000000	0.2351578	-0.0000000	0.7648422
2	0.7550000	0.0098422	0.0000000	0.7648422
3	0.7650042	0.0001619	-0.0000000	0.7648422
4	0.7648408	0.0000014	0.0000000	0.7648422
5	0.7648422	0.0000000	-0.0000000	0.7648422
6	0.7648422	0.0000000	0.0000014	0.7648408
7	0.7648422	0.0000000	-0.0001620	0.7650042
8	0.7648422	0.0000000	0.0098422	0.7550001
9	0.7648422	0.0000000	-0.2351578	1.0000000
10	0.7648422	0.0000000	0.7648422	0.0000000

Таблица 2. Значения и погрешности для cos(x).

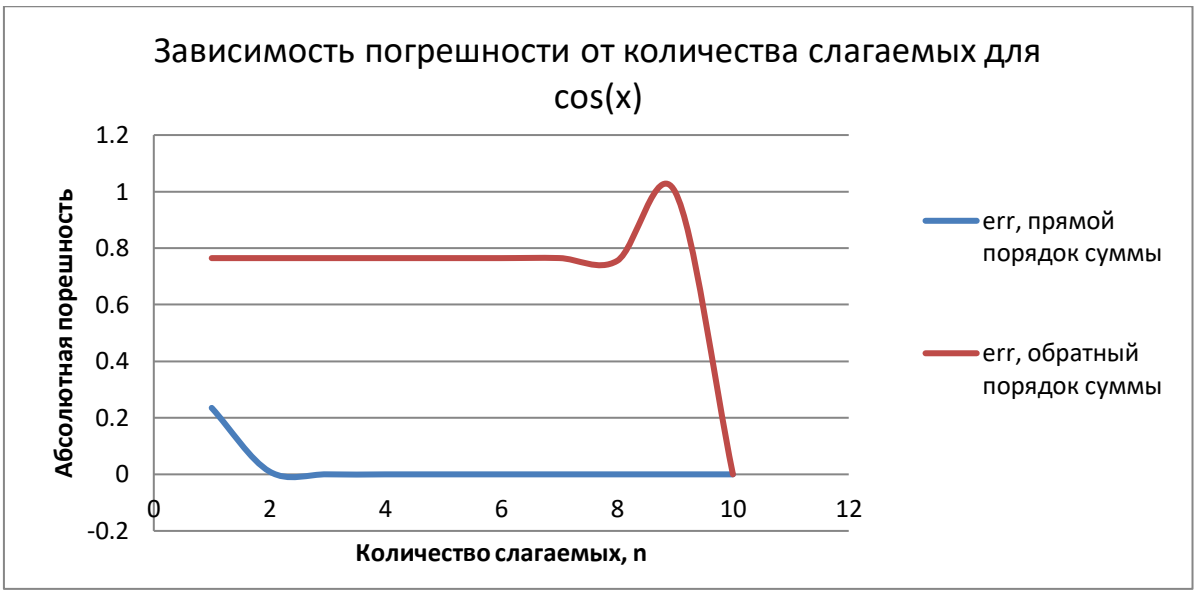


График 2. Зависимость погрешности от количества слагаемых для cos(x).

Аналогично sin(x), для функции cos(x) прямой порядок суммирования обеспечивает быстрое уменьшение абсолютной погрешности до пренебрежимо малых значений. Обратный порядок суммирования приводит к высокой начальной погрешности, которая потом имеет резкий скачок вверх.

3. Функция exp(x)

n, кол-во слагаемых	exp(x), суммы в прямом порядке	err, погрешность	exp(x), суммы в прямом порядке	err, погрешность
1	1.0000000	0.0137527	0.0000001	2.0137527
2	1.7000000	0.3137527	0.0000015	2.0137513
3	1.9450001	0.0687526	0.0000179	2.0137348
4	2.0021667	0.0115860	0.0001813	2.0135715
5	2.0121710	0.0015817	0.0015819	2.0121708
6	2.0135715	0.0001812	0.0115860	2.0021667
7	2.0137348	0.0000179	0.0687527	1.9450001
8	2.0137513	0.0000014	0.3137527	1.7000000
9	2.0137527	0.0000000	1.0137527	1.0000000
10	2.0137527	0.0000000	2.0137527	0.0000000

Таблица 3. Значения и погрешности для exp(x).

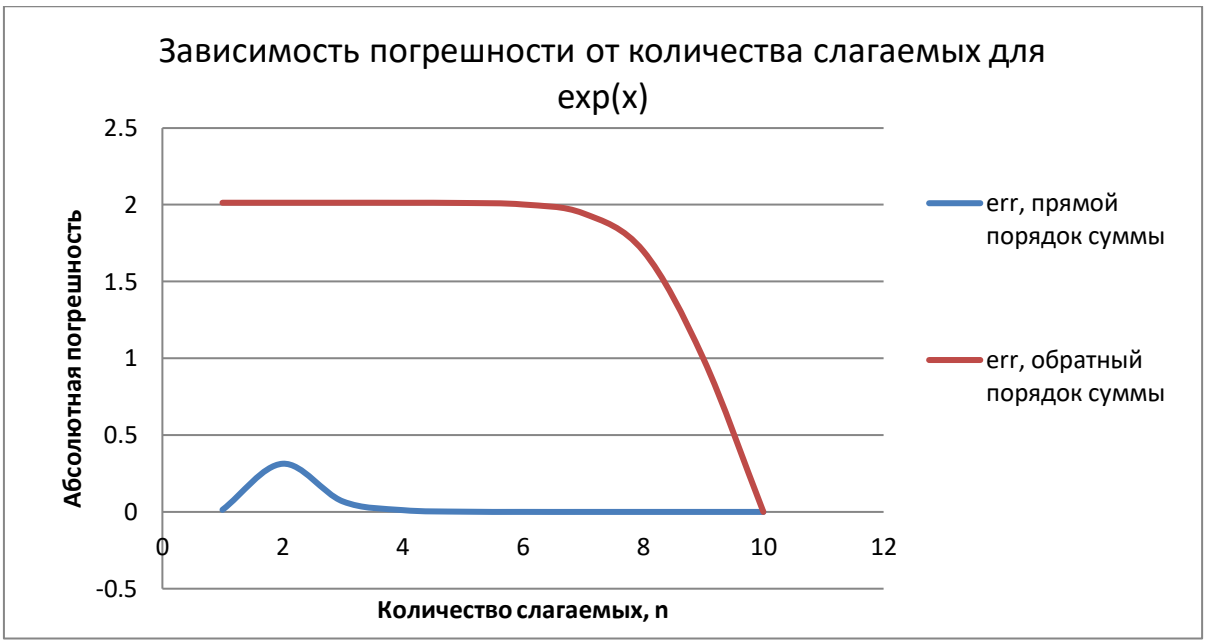


График 3. Зависимость погрешности от количества слагаемых для exp(x).

Прямой порядок суммирования быстро устремляет значение абсолютной погрешности к нулю. Обратный порядок суммирования обеспечивает высокую абсолютную погрешность.

4. Функция ln(x)

n, кол-во слагаемых	ln(x), суммы в прямом порядке	err, погрешность	ln(x), суммы в прямом порядке	err, погрешность
1	0.7000000	0.1693717	-0.0028248	0.5334530
2	0.4550000	0.0756283	0.0016590	0.5289693
3	0.5693333	0.0387051	-0.0055470	0.5361753
4	0.5093083	0.0213199	0.0062179	0.5244104
5	0.5429223	0.0122941	-0.0133903	0.5440186
6	0.5233142	0.0073141	0.0202237	0.5104045
7	0.5350791	0.0044508	-0.0398013	0.5704296
8	0.5278730	0.0027552	0.0745320	0.4560962
9	0.5323568	0.0017285	-0.1704679	0.7010962
10	0.5295321	0.0010962	0.5295321	0.0010962

Таблица 3 . Значения и погрешности для ln(x).

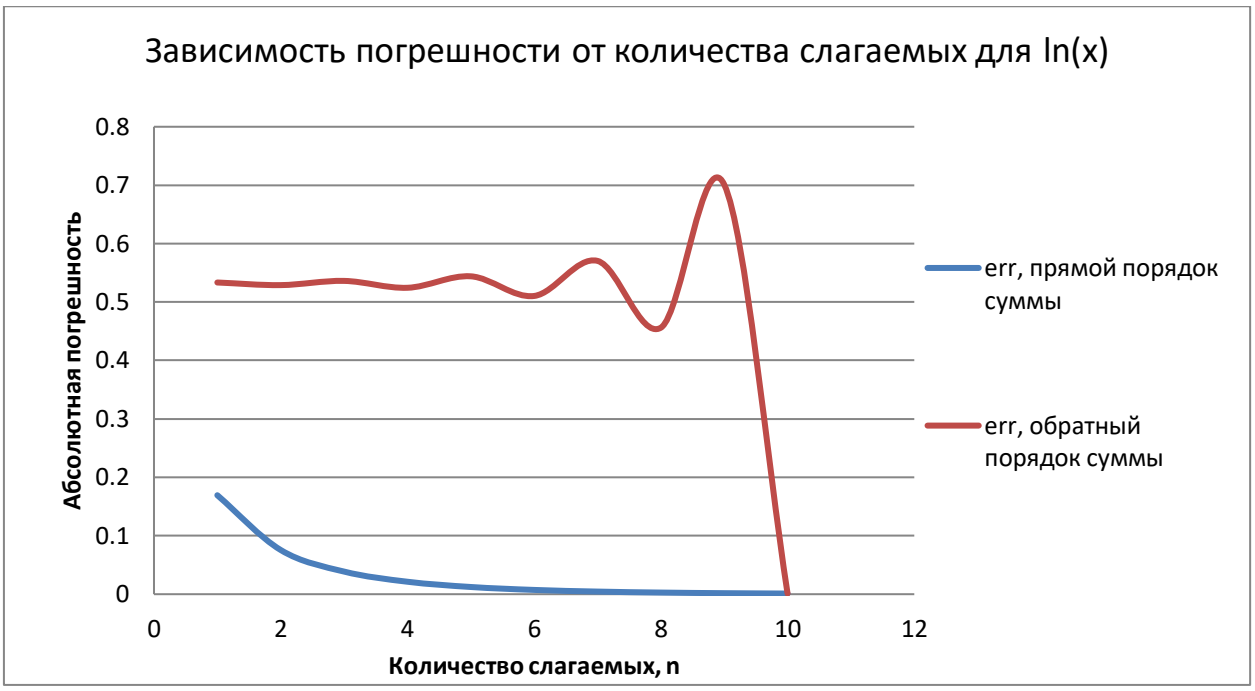


График 4. Зависимость погрешности от количества слагаемых для ln(x).

Прямой порядок суммирования обеспечивает монотонное снижение погрешности до минимума. Обратный порядок суммирования показывает высокую и нестабильную и погрешность.

Можем сделать вывод, что порядок сложения (от большего по модулю слагаемого к меньшему и наоборот) влияет на точность вычислений, особенно когда числа сильно отличаются по размеру. Различия в точности вычислений обусловлены особенностями машинной арифметики с плавающей запятой: при сложении чисел, сильно отличающихся по модулю, младшие разряды наименьших чисел могут быть потеряны.

Заключение

В работе были проведены вычисления значений функций sin(x), cos(x), exp(x), ln(x) в прямом порядке суммирования членов ряда Тейлора (слева направо, т.е. от большего по модулю к меньшему) и в обратном порядке (справа налево, т.е. от меньшего по модулю). Также построены графики зависимости абсолютной погрешности от количества слагаемых в ряде Тейлора. Можно сделать вывод, что результат сложения отличающихся по модулю чисел зависит от порядка суммирования: суммирование членов ряда Тейлора в прямом порядке (от большего слагаемого к меньшему) показало более высокую точность и более быструю сходимость к эталонному значению (вычисленному с помощью стандартных функций библиотеки "`math.h`").

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.

Приложение